SMART CLASSROOM: AI-ENABLED FACE RECOGNITION ATTENDANCE AND STUDENT BEHAVIOR ANALYTICS PLATFORM

A PROJECT REPORT

Submitted by

SABARINATHAN D (310624243149)

in partial fulfillment for the award of the Degree

of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION)

AFFILIATED TO ANNA UNIVERSITY,

CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled **"SMART CLASSROOM: AI-ENABLED FACE RECOGNITION ATTENDANCE AND STUDENT BEHAVIOR ANALYTICS PLATFORM"** is the Bonafide work of **"SABARINATHAN D (310624243149)"**, who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT
Dr. T. Kalai Selvi
Easwari Engineering College,
Ramapuram, Chennai – 600089.

SUPERVISOR
Ms. D. S. K. Harshitha
Easwari Engineering College,
Ramapuram, Chennai – 600089.

EASWARI ENGINEERING COLLEGE

Certificate Of Viva-Voce Examination

This is to certify that the Viva Voce Examination of the Project Work entitled “Smart Classroom: AI-Enabled Face Recognition Attendance and Student Behavior Analytics Platform” submitted by Sabarinathan D (310624243149) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology was conducted on _____ in the Department of Artificial Intelligence and Data Science, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Date : 27.10.2026

ABSTRACT

Classroom attendance and student engagement are still tracked manually in most Indian colleges. A faculty member calls out roll numbers at the start of every hour, writes them into a register, and later copies the data into a spreadsheet or a college portal. This consumes several minutes of every lecture, invites proxy attendance, and produces records that are rarely analysed. Student behavior during the lecture, such as whether learners are attentive, talking, sleeping or using their phones, is not recorded at all, so teachers have no objective evidence of engagement when they review a class or counsel a student.

This project presents **Smart Classroom**, a full-stack, AI-enabled attendance and behavior analytics platform. The system is built with a React 18 single page application (Vite), a Node.js and Express REST API, a MongoDB database running on persistent storage, and a separate Python computer-vision service. The vision service captures the classroom camera feed with OpenCV, detects faces with MTCNN, converts each face into a 512-dimensional FaceNet embedding and matches it against the enrolled students of the selected classroom using cosine similarity. A YOLOv8 pose and object model, combined with facial cues, classifies each recognised student into one of six behaviors: Studying, Speaking, Sleeping, Phone Use, Raising Hand and Distracted.

The web portal provides six modules: Dashboard, Live AI Vision, Classrooms, Students, Attendance and Behavior. With one click on *Auto-Fill From Camera*, the teacher receives a face-verified attendance roster which can be reviewed and saved as an attendance session with present, absent and late counts and an attendance percentage. Behavior snapshots are stored as timestamped sessions, and a per-student behavior summary aggregates every logged observation. During testing on a live classroom feed, the system identified the enrolled students with verified face matches, reported behavior labels with confidence scores of 96–98%, streamed an annotated HUD video at about 2 frames per second on a CPU-only laptop, and synchronised live status with the database every 10 seconds. The result is a practical, low-cost tool that reduces attendance time to seconds and gives teachers an evidence-based view of classroom engagement.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal **Dr. P. Deiva Sundari** and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department **Dr. T. Kalai Selvi**, Department of Artificial Intelligence and Data Science, for the continuous encouragement and support.

We are extremely thankful to our Project Guide **Ms. D. S. K. Harshitha**, Department of Artificial Intelligence and Data Science, for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Artificial Intelligence and Data Science for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**i**
	ACKNOWLEDGEMENT	**ii**
	LIST OF TABLES	**v**
	LIST OF FIGURES	**vi**
1.	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	2
	1.3 OBJECTIVE	2
	1.4 SCOPE OF THE PROJECT	3
	1.5 MOTIVATION	4
2.	**LITERATURE SURVEY**	**5**
	2.1 LITERATURE REVIEW	5
	2.2 EXISTING SYSTEM	6
	2.3 PROPOSED SYSTEM	6
3.	**SYSTEM ANALYSIS**	**8**
	3.1 SYSTEM REQUIREMENTS	8
	3.1.1 Functional Requirements	8
	3.1.2 Non-Functional Requirements	9
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	9
4.	**METHODOLOGY**	**11**
	4.1 SYSTEM ARCHITECTURE	11
	4.2 MODULE DESCRIPTION	12
	4.3 CLASS DIAGRAM	14
	4.4 SEQUENCE DIAGRAM	14
5.	**RESULTS AND DISCUSSION**	**16**
	5.1 IMPLEMENTATION OVERVIEW	16
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	16
	5.2.1 FaceNet Enrollment & Recognition	16
	5.2.2 YOLOv8 Behavior Classification	18
	5.2.3 Attendance Session Recording	19
	5.2.4 Behavior Snapshot & Summary Aggregation	20

CHAPTER NO.	TITLE	PAGE No.
	5.2.5 JWT Authentication & Role Guards	21
	5.3 TESTING AND OBSERVED RESULTS	21
6.	**CONCLUSION & FUTURE WORK**	**23**
	6.1 CONCLUSION	23
	6.2 FUTURE ENHANCEMENTS	23
	APPENDIX: SOURCE CODE	**25**
	SCREENSHOTS OF THE PROJECT	**35**
	REFERENCES	**38**

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE No.
3.1	Minimum Software Requirements for Development and Deployment	9
3.2	Minimum and Recommended Hardware Specifications	10
4.1	Summary of Smart Classroom Modules and Core Responsibilities	13
5.1	Functional Test Cases and Observed Results	22

LIST OF FIGURES

FIGURE NO.	FIGURE TITLE	PAGE No.
4.1	Smart Classroom -- Layered System Architecture	12
4.2	UML Class Diagram of Smart Classroom MongoDB Document Models	14
4.3	AI Attendance & Behavior Capture Sequence Diagram	15
1	Live Computer Vision Camera -- FaceNet + YOLOv8 HUD Stream with Live Student Status	35
2	Classroom Management -- Add Classroom Form and Enrolled Classroom List	35
3	Student Management -- Student Registration and Classroom Assignment	36
4	Live Computer Vision Attendance -- Face-Verified Roster and Recent Attendance Sessions	36
5	Automated AI Behavior Monitoring -- Camera-Detected Behaviors with Confidence Scores	37
6	Recent Behavior Sessions & Logs and Per-Student Behavior Summary	37

CHAPTER 1
INTRODUCTION

This chapter deals with the introduction, problem definition, objectives, scope and motivation behind Smart Classroom, an AI-enabled face recognition attendance and student behavior analytics platform.

1.1 OVERVIEW

Attendance is one of the most routine and most time-consuming activities in a college classroom. In a typical lecture hour of fifty minutes, a teacher spends five to ten minutes calling roll numbers and marking a paper register. Across six or seven hours a day and an entire semester, this adds up to many lecture hours lost to administration. The manual process also invites proxy attendance, where one student answers for an absent friend, and the resulting records are often transcribed late and with errors.

Beyond attendance, teachers have very little objective information about what actually happens in the classroom while they teach. Whether students are attentive, talking among themselves, sleeping or using mobile phones is judged only by the teacher's impression at that moment. Nothing is recorded, so there is no data to support a discussion with a student, a mentor meeting or a review of teaching methods.

Smart Classroom addresses both problems with a single camera and a web portal. The platform combines a React 18 single page application, a Node.js and Express REST API, a MongoDB database and a Python computer-vision service. The vision service recognises enrolled students from the live classroom feed using FaceNet face embeddings and classifies their behavior using a YOLOv8 based model. Teachers can fill the attendance roster from the camera in one click, save behavior snapshots during the lecture, and review session logs and per-student behavior summaries afterwards.

By turning a camera that most smart classrooms already have into a source of structured attendance and engagement data, the project shows how computer vision can reduce administrative work and give teachers evidence-based insight into their classes.

1.2 PROBLEM DEFINITION

Educational institutions that rely on manual or semi-manual classroom processes face three connected problems:

1. **Time-Consuming and Error-Prone Attendance:** Roll calls and paper registers consume a significant part of every lecture. Records are copied into spreadsheets or portals later, which introduces transcription mistakes and delays. Proxy attendance is difficult to detect because the teacher cannot verify every voice against a face.
2. **No Objective Record of Student Engagement:** Teachers have no systematic way to record whether students were studying, talking, sleeping, distracted or using phones. Engagement is judged subjectively and is forgotten once the lecture ends, so patterns across days or weeks are never visible.
3. **Disconnected and Hard-to-Analyse Data:** Even when attendance is digitised, it is stored separately from any information about classroom behavior. There is no single place where a teacher or mentor can see a student's presence and conduct together, session by session.

1.3 OBJECTIVE

The overall goal of the project is to design, develop and test an AI-enabled web platform that automates classroom attendance and records student behavior from a live camera feed. The specific objectives are:

- **Face Recognition Attendance:** Detect faces with MTCNN, generate FaceNet embeddings and match them against the enrolled students of the selected classroom to mark presence automatically, with a visible "Face Verified" indicator for each recognised student.
- **Automated Behavior Monitoring:** Classify each recognised student into Studying, Speaking, Sleeping, Phone Use, Raising Hand or Distracted using YOLOv8 pose keypoints, phone detection and facial cues, and report a confidence score for every label.
- **Session-Based Records:** Store every attendance and behavior capture as a timestamped session with per-student detail, totals and attendance percentage, so that past classes can be inspected at any time.

- **Analytics for Teachers:** Aggregate behavior logs into a per-student summary showing how many times each behavior was observed, giving teachers objective data on engagement.
- **Simple Management Portal:** Provide classroom and student management (create, list, assign and delete) through a clean, responsive React interface with secure JWT based login.
- **Live Monitoring:** Stream an annotated MJPEG video with a heads-up display (present count, behaviors in frame, FPS) and synchronise live status with the database every 10 seconds.

1.4 SCOPE OF THE PROJECT

The scope of the platform is divided into implemented deliverables and deliberate boundaries:

A. In-Scope Deliverables:

1. **Dashboard:** Overview of classrooms, students and recent sessions.
2. **Live AI Vision:** Classroom selector, live FaceNet + YOLOv8 HUD stream, present count, AI vision FPS, faces in frame, and live per-student presence and behavior cards with Pause Camera, Sync DB Now and Enroll Face actions.
3. **Classroom Management:** Add classrooms with name, section and subject; list them with the assigned teacher and enrolled count; delete classrooms.
4. **Student Management:** Add students with roll number, full name, email and assigned classroom; list and delete students; enroll a student's face from the camera.
5. **AI Attendance:** Auto-fill Present/Absent/Late from the camera, manual override, save attendance sessions and inspect each session's roster with the recording method.
6. **AI Behavior Monitoring:** Auto-fill camera-detected behaviors with confidence, save behavior snapshots, view recent behavior sessions and a per-student behavior summary.
7. **Security:** Admin login with hashed passwords, JWT protected APIs and role-based access.

B. Out-of-Scope Boundaries:

1. **Multi-Camera and Large-Hall Coverage:** The current version processes one camera per classroom; stitching several cameras for large auditoriums is reserved for future work.
2. **Emotion and Academic Performance Prediction:** The system records observable behavior only; it does not infer emotions or predict marks.
3. **Integration with University ERP:** Export to the official university attendance portal is handled manually in this version.

1.5 MOTIVATION

The motivation for Smart Classroom came from our own experience as students. Every lecture begins with a roll call, and every semester ends with attendance shortages that surprise students because they never saw their records until it was too late. Teachers, on the other hand, often know which students are disengaged but have no evidence to discuss it with them or with their mentors.

Recent progress in deep learning has made face recognition and real-time object and pose detection accurate enough to run on an ordinary laptop. Pre-trained models such as FaceNet and YOLOv8 are openly available, and a modern JavaScript stack makes it straightforward to build a responsive portal around them. Combining these technologies gives colleges a low-cost way to save lecture time, eliminate proxy attendance and give teachers a clearer, data-driven view of how their classes are going.

CHAPTER 2
LITERATURE SURVEY

This chapter deals with a survey of research on face recognition, object and pose detection and student engagement analysis, reviews the attendance systems commonly used in colleges today, and introduces the proposed Smart Classroom system.

2.1 LITERATURE REVIEW

Early automated face detection became practical with the boosted cascade of Haar-like features proposed by Viola and Jones [1], which made real-time frontal face detection possible on ordinary hardware. However, cascade detectors are sensitive to pose and lighting, which limits their reliability in a classroom where students turn their heads and lighting changes through the day. Zhang et al. [2] introduced the Multi-task Cascaded Convolutional Network (MTCNN), which jointly detects faces and aligns facial landmarks through three cascaded CNN stages and is considerably more robust to pose variation.

For identity recognition, Schroff, Kalenichenko and Philbin [3] proposed FaceNet, which maps each face image to a compact Euclidean embedding trained with a triplet loss so that distances between embeddings directly reflect face similarity. Recognition then reduces to a simple nearest-neighbour comparison, which suits attendance systems because new students can be enrolled without retraining the network. Parkhi, Vedaldi and Zisserman [4] and the VGGFace2 dataset of Cao et al. [5] further showed that deep networks trained on large, diverse face datasets generalise well across pose and age. The FaceNet model used in this project is an Inception-ResNet network [6] pre-trained on VGGFace2.

For behavior analysis, Redmon et al. [7] introduced YOLO, which treats object detection as a single regression problem and runs in real time. The Ultralytics YOLOv8 family [8] extends this with detection and human pose estimation models that output body keypoints such as shoulders and wrists, similar in purpose to the part-affinity approach of OpenPose [9]. These keypoints make it possible to recognise postures such as a raised hand or a dropped head. Soukupová and Čech [10] showed that the eye aspect ratio computed from facial landmarks is a reliable signal for detecting closed eyes, which we

use as a cue for sleeping.

Whitehill et al. [11] demonstrated that student engagement can be recognised automatically from facial expressions with accuracy comparable to human observers, supporting the idea that camera-based engagement analysis is meaningful in an educational setting. Together, these works provide the building blocks for a system that both identifies students and describes what they are doing during a lecture.

2.2 EXISTING SYSTEM

Attendance and engagement tracking in most colleges today falls into three categories, each with clear limitations:

1. **Manual Roll Call and Paper Registers:** The teacher calls each roll number and marks the register. It needs no equipment but wastes lecture time, allows proxy attendance and must be re-entered into a digital system later, often with errors.
2. **Biometric Devices and ID Cards:** Fingerprint scanners and RFID cards speed up entry but create queues at the door, require physical contact or card handling, and only confirm that a student entered the room, not that they stayed for the lecture. Cards can also be handed to another student.
3. **Attendance-Only Software:** Mobile or web apps let teachers tick names on a screen. They digitise the register but still depend on the teacher's manual input and record nothing about behavior or engagement during the class.

2.3 PROPOSED SYSTEM

The proposed Smart Classroom platform replaces these approaches with a contactless, camera-based system that records both presence and behavior. Its key features are:

1. **Contactless Face Recognition Attendance:** Students are recognised from the classroom camera using MTCNN and FaceNet embeddings. The teacher fills the roster from the camera in one click and can still override any entry before saving.
2. **Continuous Behavior Monitoring:** A YOLOv8 based pipeline labels each recognised student as Studying, Speaking, Sleeping, Phone Use, Raising Hand or Distracted with a confidence score, throughout the lecture rather than only at its start.
3. **Session Logs and Analytics:** Every capture is stored as a timestamped session in MongoDB. Teachers can expand any session to see each student's status and review a

per-student behavior summary across sessions.

4. **Modular Full-Stack Architecture:** A React portal, an Express REST API and an independent Python vision service communicate over HTTP, so the AI component can be upgraded or moved to a GPU machine without changing the portal.

CHAPTER 3
SYSTEM ANALYSIS

This chapter deals with the system analysis of the Smart Classroom platform, specifying the functional and non-functional requirements and the software and hardware environment needed for development and deployment.

3.1 SYSTEM REQUIREMENTS

Requirements analysis defines what the platform must do, how well it must perform, and the quality standards it must meet.

3.1.1 Functional Requirements

- **FR-1: Authentication:** The system must authenticate users with email and password, store passwords as bcrypt hashes, issue a signed JSON Web Token on login and protect every API route with token verification and role checks (Admin, Teacher).
- **FR-2: Classroom Management:** The system must allow creating a classroom with name, section and subject, list classrooms with their teacher and enrolled student count, and delete a classroom.
- **FR-3: Student Management:** The system must allow adding a student with a unique roll number, full name, email and optional classroom, listing students, deleting students, and enrolling a student's face from the camera.
- **FR-4: Live Computer Vision:** The system must stream the classroom camera as an annotated MJPEG feed, show the present count, faces in frame and processing FPS, and display each student's live presence and behavior.
- **FR-5: AI Attendance:** The system must auto-fill Present/Absent/Late for every student of the selected classroom from face recognition, allow manual changes, and save the result as an attendance session with totals and percentage.
- **FR-6: AI Behavior Monitoring:** The system must auto-fill one of six behaviors for each recognised student with a confidence score, allow manual correction, and save behavior snapshots as sessions.
- **FR-7: Session History:** The system must list recent attendance and behavior sessions and expand any session to show each student's roll number, name, class, status or

behavior, and recording method.

- **FR-8: Behavior Summary:** The system must aggregate all behavior records per student and show counts for Studying, Speaking, Sleeping, Phone Use, Raising Hand and Distracted.

3.1.2 Non-Functional Requirements

- **NFR-1: Performance:** Standard API reads must respond within about 200 ms on a local network. The vision service must process at least 2 frames per second on a CPU-only laptop and synchronise live status with the database every 10 seconds.
- **NFR-2: Accuracy:** A face is accepted as a match only when its cosine similarity with the enrolled embedding is at least 0.70; otherwise it is treated as unknown, which keeps false attendance low.
- **NFR-3: Security and Privacy:** Only face embeddings (numeric vectors) are stored, not raw face images. Passwords are hashed with bcrypt, APIs are protected by JWT, and Helmet headers and a restricted CORS origin are enabled.
- **NFR-4: Reliability:** MongoDB runs on a persistent disk so that sessions survive restarts, and the portal shows the database connection status in its header at all times.
- **NFR-5: Usability:** The interface must be simple enough for a teacher to take attendance in a few clicks, with clear colour-coded badges for presence and behavior, and must work on desktop and tablet screens.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

The software and hardware specifications used for development and deployment are listed in Tables 3.1 and 3.2.

Table 3.1: Minimum Software Requirements for Development and Deployment

Software Layer	Component / Technology	Version	Role in Smart Classroom
Operating System	Windows 11 / Ubuntu Linux	64-bit	Development and hosting OS
Frontend	React + Vite	React 18 / Vite 5	Single page portal (port 5173)
Backend Runtime	Node.js + Express	Node 20 LTS / Express 4	REST API, auth, business logic

Software Layer	Component / Technology	Version	Role in Smart Classroom
Database	MongoDB + Mongoose	MongoDB 7 / Mongoose 8	Persistent document storage (port 27017)
AI Service	Python + Flask	Python 3.10 / Flask 3	Vision service and MJPEG stream (port 5001)
Computer Vision	OpenCV	4.x	Camera capture, frame processing, HUD drawing
Face Recognition	facenet-pytorch (MTCNN + InceptionResnetV1)	2.5	Face detection and 512-d embeddings
Behavior Model	Ultralytics YOLOv8	8.x (n-pose, n)	Pose keypoints and phone detection
Deep Learning	PyTorch	2.x	Model inference runtime
Security	jsonwebtoken, bcryptjs, helmet	9.x / 2.x / 7.x	Tokens, hashing, secure headers

Table 3.2: Minimum and Recommended Hardware Specifications

Hardware Component	Minimum Specification	Recommended Specification
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (4 cores)	Intel Core i7 / AMD Ryzen 7 (8 cores)
GPU	Not required (CPU inference)	NVIDIA GPU with CUDA, 4 GB VRAM or more
Memory (RAM)	8 GB	16 GB
Storage	256 GB SSD (10 GB free)	512 GB NVMe SSD
Camera	720p USB webcam / laptop camera	1080p wide-angle classroom camera
Network	Local Wi-Fi / LAN	Wired LAN, 100 Mbps
Display	1366 x 768	1920 x 1080

CHAPTER 4
METHODOLOGY

This chapter deals with the design of Smart Classroom, describing the layered system architecture, the functional modules, the UML class diagram of the MongoDB document models and the sequence of the AI attendance and behavior capture workflow.

4.1 SYSTEM ARCHITECTURE

Smart Classroom follows a layered architecture that separates the user interface, the API, business logic, the AI vision pipeline and data storage. As shown in Figure 4.1, the platform has five tiers:

1. **Presentation Layer:** A React 18 single page application built with Vite and served on port 5173. It provides the Dashboard, Live AI Vision, Classrooms, Students, Attendance and Behavior pages, and shows the database connection status and the logged-in user in a fixed header.
2. **API and Security Layer:** A Node.js and Express REST API that verifies JWT tokens, enforces Admin and Teacher roles, validates input and applies Helmet and CORS protection.
3. **Business Logic Layer:** Services for classrooms, student enrollment, attendance sessions, behavior snapshots and analytics aggregation.
4. **AI Vision Layer:** A Python Flask service on port 5001 that reads the camera with OpenCV, runs MTCNN and FaceNet for recognition and YOLOv8 for behavior, and serves an annotated MJPEG stream.
5. **Data Persistence Layer:** MongoDB on port 27017 with a persistent disk, holding classrooms, students, face embeddings and attendance and behavior session collections.

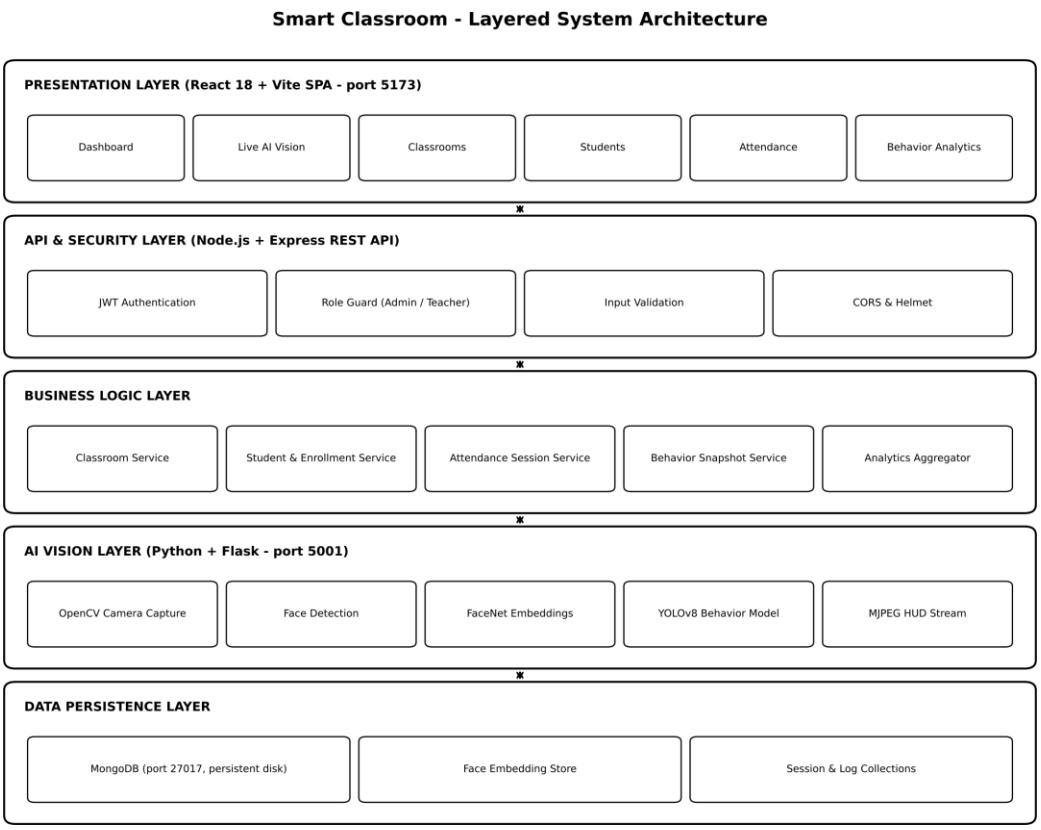


Figure 4.1: Smart Classroom -- Layered System Architecture

4.2 MODULE DESCRIPTION

The platform is organised into six functional modules that work together:

4.2.1 Dashboard Module: Gives a quick overview of the system: number of classrooms and students, the latest attendance and behavior sessions, and navigation to every module from the sidebar.

4.2.2 Live AI Vision Module: Lets the teacher select the active classroom and watch the live “FaceNet + YOLOv8 HUD” stream. The header shows students present (for example 2 / 2, 100%), AI vision FPS and faces currently in frame. A side panel lists every student of the classroom with a PRESENT/ABSENT badge and the latest behavior. Buttons allow pausing the camera, forcing a database sync and enrolling a student’s face.

4.2.3 Classroom Management Module: Creates classrooms with a name (for example AI & Data Science – 3rd Year), section and subject, and lists them with the teacher, the number of enrolled students and a delete action.

4.2.4 Student Management Module: Registers students with a unique roll number, full name, email and an optional assigned classroom, and lists them with their classroom and a delete action. Face enrollment links each student to a stored embedding.

4.2.5 AI Attendance Module: Shows the roster of the selected classroom with a "Face Verified" tag and the camera AI status of each student. *Auto-Fill From Camera* sets Present, Absent or Late; the teacher can adjust and click *Save Attendance Record*. Recent attendance sessions show date and time, present, absent and late counts and attendance percentage, and each session expands into a roster showing the recorded method (AI Face Recognition / Vision or manual).

4.2.6 AI Behavior Monitoring Module: Displays the camera-detected behavior and confidence for each student (for example SPEAKING 98%, STUDYING 96%) with a radio selector for the six behaviors. *Save Behavior Snapshot* stores a session; Recent Behavior Sessions & Logs list each capture with its primary behaviors, and the Behavior Summary counts every behavior observed for each student, with a link to the student's individual profile.

Table 4.1: Summary of Smart Classroom Modules and Core Responsibilities

Module Code	Module Name	Primary Entities	Key Responsibility
MOD-01	Dashboard	Classroom, Session	Overview and navigation
MOD-02	Live AI Vision	FaceEmbedding, Student	Live HUD stream, presence and behavior status, face enrollment
MOD-03	Classrooms	Classroom, User	Create, list and delete classrooms
MOD-04	Students	Student	Register, assign and delete students
MOD-05	Attendance	AttendanceSession, AttendanceRecord	Camera auto-fill, save sessions, roster history
MOD-06	Behavior	BehaviorSession, BehaviorRecord	Behavior detection, snapshots, per-student summary

4.3 CLASS DIAGRAM

The data model is implemented as MongoDB collections defined with Mongoose schemas. Figure 4.2 shows the main documents, their attributes and the references between them. A User (admin or teacher) owns many Classrooms; a Classroom has many Students; each Student has one FaceEmbedding holding the 512-dimensional reference vector. Each attendance capture creates one AttendanceSession with many AttendanceRecords, and each behavior capture creates one BehaviorSession with many BehaviorRecords linked to individual students.

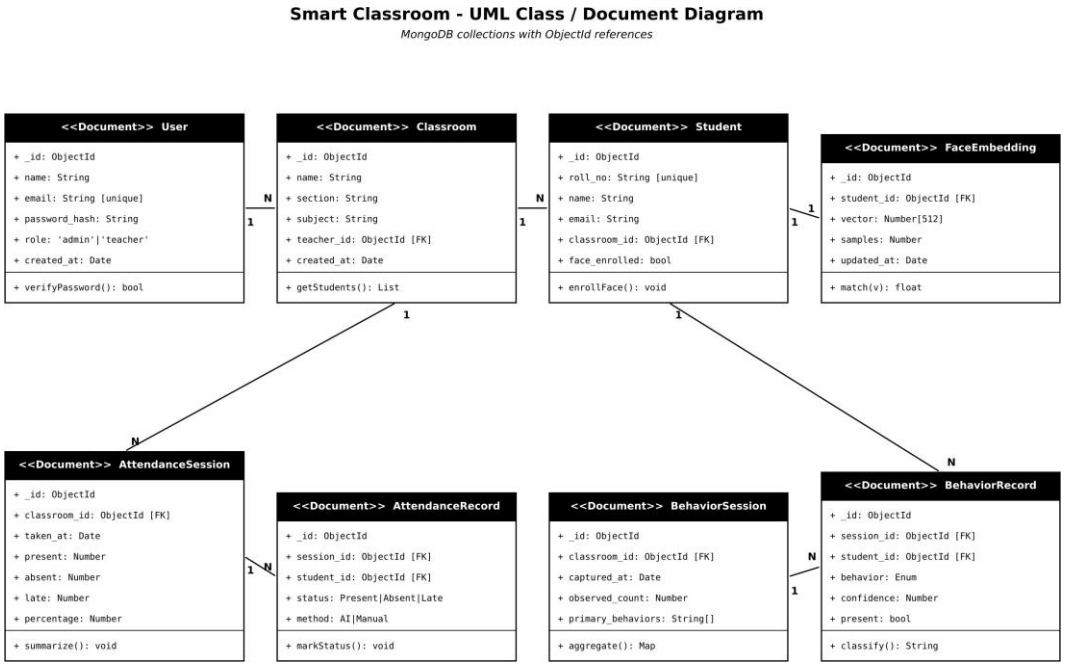


Figure 4.2: UML Class Diagram of Smart Classroom MongoDB Document Models

4.4 SEQUENCE DIAGRAM

Figure 4.3 shows the runtime flow when a teacher fills attendance from the camera. The React portal calls the Express API, which requests current presence from the AI vision service. The vision service reads frames from the camera, detects faces, computes FaceNet embeddings, matches them against the classroom gallery and classifies behavior with YOLOv8. The result is returned to the portal to pre-fill the roster. When the teacher saves, the API validates the students against the classroom, inserts the

AttendanceSession and its AttendanceRecords into MongoDB, and returns the saved session with the attendance percentage. Behavior snapshots follow the same pattern.

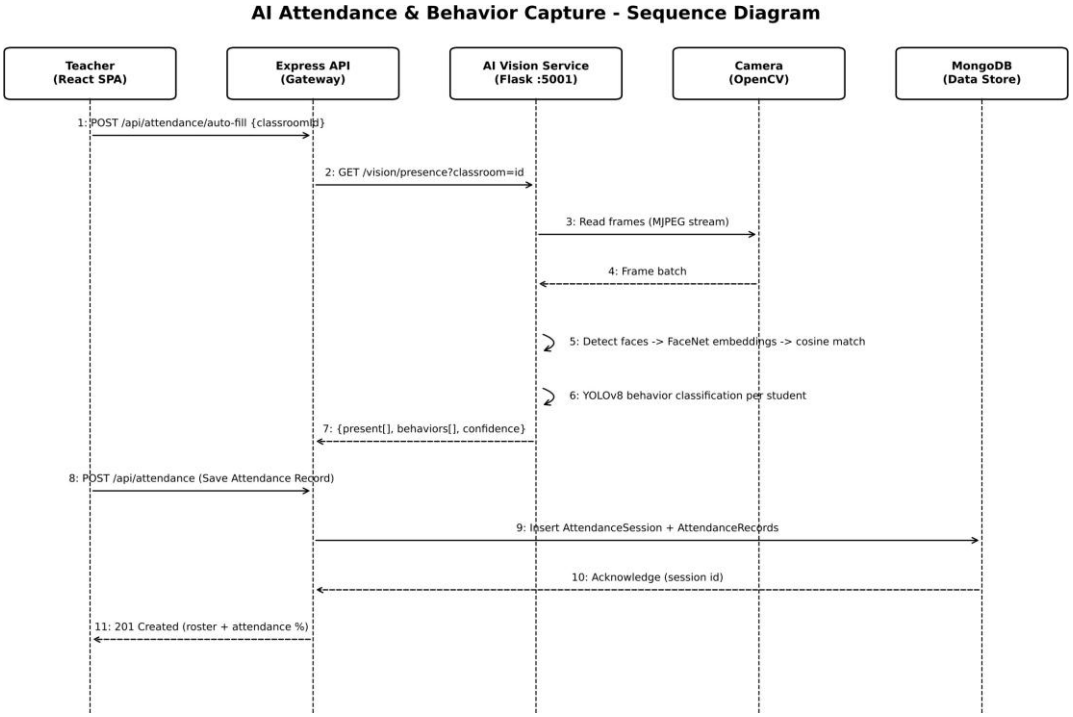


Figure 4.3: AI Attendance & Behavior Capture Sequence Diagram

CHAPTER 5
RESULTS AND DISCUSSION

This chapter deals with the implementation of Smart Classroom, explains the key functions with code walkthroughs for face recognition, behavior classification, attendance recording, behavior analytics and authentication, and discusses the observed results.

5.1 IMPLEMENTATION OVERVIEW

The project is organised into three cooperating applications: a **frontend** (React 18 + Vite), a **backend** (Node.js + Express + Mongoose) and an **ai_service** (Python + Flask + PyTorch). They communicate over HTTP on ports 5173, 5000 and 5001 respectively, and share a MongoDB database on port 27017.

Frontend: Built with functional React components and React Router. An Axios client attaches the JWT to every request. The Live AI Vision page embeds the MJPEG stream from the vision service and polls live status every 10 seconds. Attendance and Behavior pages render the classroom roster with radio selectors, and session tables expand inline to show each student's detail.

Backend: Built with Express, organised into models, controllers, middleware and routes. Mongoose schemas define the collections; controllers implement classroom and student CRUD, attendance and behavior sessions, and MongoDB aggregation pipelines for summaries.

AI Service: A background thread continuously reads frames with OpenCV, runs MTCNN and FaceNet for identity and YOLOv8 for behavior, keeps the latest state in memory and draws a heads-up display on each frame. Flask exposes the annotated frames as an MJPEG stream and the latest presence and behavior as JSON.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

The core functions of the platform are explained below with code walkthroughs.

5.2.1 FaceNet Enrollment & Recognition

During enrollment, several frames of the student are captured; MTCNN crops and aligns the face and FaceNet converts it into a 512-dimensional vector. The vectors are

averaged and L2-normalised into one reference embedding stored in MongoDB. During recognition, every face in the frame is embedded the same way and compared with the classroom gallery. Because all vectors are normalised, the dot product equals the cosine similarity, and a match is accepted only above the 0.70 threshold:

```
# ai_service/recognition.py - FaceNet Enrollment & Recognition
import numpy as np
import torch
from facenet_pytorch import MTCNN, InceptionResnetV1

device = "cuda" if torch.cuda.is_available() else "cpu"
mtcnn = MTCNN(keep_all=True, device=device)          # face detector
facenet = InceptionResnetV1(pretrained="vggface2").eval().to(device)

MATCH_THRESHOLD = 0.70   # minimum cosine similarity for a positive match


def embed_faces(frame_rgb):
    """Detect every face in a frame and return (boxes, 512-d embeddings)."""
    boxes, _ = mtcnn.detect(frame_rgb)
    if boxes is None:
        return [], []
    faces = mtcnn.extract(frame_rgb, boxes, save_path=None).to(device)
    with torch.no_grad():
        vectors = facenet(faces).cpu().numpy()
    vectors /= np.linalg.norm(vectors, axis=1, keepdims=True)  # L2-normalise
    return boxes, vectors


def enroll_student(student_id, frames, db):
    """Average embeddings from several frames into one reference vector."""
    samples = [embed_faces(f)[1][0] for f in frames if len(embed_faces(f)[1])]
    if not samples:
        raise ValueError("No face found in enrollment frames")
    ref = np.mean(samples, axis=0)
```

```
    ref /= np.linalg.norm(ref)
    db.face_embeddings.update_one(
        {"student_id": student_id},
        {"$set": {"vector": ref.tolist(), "samples": len(samples)}},
        upsert=True)
    db.students.update_one({"_id": student_id},
                           {"$set": {"face_enrolled": True}})


def identify(vectors, gallery):
    """Match each detected face against the classroom gallery."""
    ids = list(gallery.keys())
    refs = np.array([gallery[i] for i in ids])        # (N, 512)
    results = []
    for v in vectors:
        sims = refs @ v                                # cosine similarity
        best = int(np.argmax(sims))
        if sims[best] >= MATCH_THRESHOLD:
            results.append((ids[best], float(sims[best])))
        else:
            results.append((None, float(sims[best])))  # unknown face
    return results
```

5.2.2 YOLOv8 Behavior Classification

Behavior is classified by fusing YOLOv8 pose keypoints with a YOLOv8 phone detector and facial ratios. The rules are checked in priority order: a wrist above the shoulder line means Raising Hand; a phone box overlapping a wrist means Phone Use; eyes closed for 15 consecutive frames or a dropped head means Sleeping; a fluctuating mouth aspect ratio means Speaking; a head turned far from the shoulder axis means Distracted; otherwise the student is Studying. The detector confidence is returned with each label and shown in the portal (for example SPEAKING 98%).

```
# ai_service/behavior.py - YOLOv8 Pose + Facial Cue Behavior Classifier
import numpy as np
from ultralytics import YOLO

pose_model = YOLO("yolov8n-pose.pt")   # 17 COCO body keypoints per person
obj_model = YOLO("yolov8n.pt")         # COCO objects (class 67 = cell phone)

EAR_SLEEP = 0.21     # eye aspect ratio below this for N frames -> sleeping
MAR_SPEAK = 0.35     # mouth aspect ratio variance threshold -> speaking
SLEEP_FRAMES = 15


def classify_person(kp, phone_boxes, ear, mar_var, closed_frames):
    """Rule-based fusion of pose keypoints and facial ratios."""
    nose, l_sh, r_sh = kp[0], kp[5], kp[6]
    l_wr, r_wr = kp[9], kp[10]
    shoulder_y = min(l_sh[1], r_sh[1])

    # 1. Raising hand: a wrist clearly above the shoulder line
    if min(l_wr[1], r_wr[1]) < shoulder_y - 40:
        return "RAISING_HAND"
    # 2. Phone use: a detected phone overlaps the wrist region
    for (x1, y1, x2, y2) in phone_boxes:
        for wx, wy in (l_wr[:2], r_wr[:2]):
            if x1 - 30 < wx < x2 + 30 and y1 - 30 < wy < y2 + 30:
                return "PHONE_USE"
    # 3. Sleeping: eyes closed for consecutive frames or head dropped
    if closed_frames >= SLEEP_FRAMES or nose[1] > shoulder_y + 25:
        return "SLEEPING"
    # 4. Speaking: mouth opening fluctuates across recent frames
```

```
    if mar_var > MAR_SPEAK:
        return "SPEAKING"
    # 5. Distracted: head turned far from the board (nose off shoulder axis)
    mid_x = (l_sh[0] + r_sh[0]) / 2
    if abs(nose[0] - mid_x) > 0.45 * abs(l_sh[0] - r_sh[0]):
        return "DISTRACTED"
    return "STUDYING"


def analyse_frame(frame, tracker_state):
    poses = pose_model(frame, verbose=False)[0]
    objs = obj_model(frame, classes=[67], verbose=False)[0]
    phones = objs.boxes.xyxy.cpu().numpy().tolist()
    out = []
    for i, kp in enumerate(poses.keypoints.xy.cpu().numpy()):
        st = tracker_state.get(i, {"ear": 0.3, "mar_var": 0.0, "closed": 0})
        label = classify_person(kp, phones, st["ear"], st["mar_var"],
                                st["closed"])
        conf = float(poses.boxes.conf[i])
        out.append({"index": i, "behavior": label, "confidence": conf})
    return out
```

5.2.3 Attendance Session Recording

When the teacher saves attendance, the controller keeps only students enrolled in the selected classroom, counts Present, Absent and Late, computes the attendance percentage as (Present + Late) / Total x 100, and stores one session document plus one record per student with the recording method:

```
// backend/controllers/attendanceController.js - Save Attendance Session
const AttendanceSession = require("../models/AttendanceSession");
const AttendanceRecord = require("../models/AttendanceRecord");
const Student = require("../models/Student");

exports.saveAttendance = async (req, res) => {
  try {
    const { classroomId, records } = req.body;   // [{studentId, status, method}]
    if (!classroomId || !Array.isArray(records) || !records.length) {
      return res.status(400).json({ success: false,
        error: "Classroom and attendance records are required." });
    }
    // Only accept students that are enrolled in this classroom
    const enrolled = await Student.find({ classroom_id: classroomId })
                                  .select("_id").lean();
    const valid = new Set(enrolled.map(s => String(s._id)));
    const clean = records.filter(r => valid.has(String(r.studentId)));

    const count = s => clean.filter(r => r.status === s).length;
    const present = count("Present"), absent = count("Absent");
    const late = count("Late");
    const percentage = Math.round(((present + late) / clean.length) * 100);

    const session = await AttendanceSession.create({
      classroom_id: classroomId, taken_at: new Date(),
      present, absent, late, percentage, taken_by: req.user.id
    });
    await AttendanceRecord.insertMany(clean.map(r => ({
      session_id: session._id, student_id: r.studentId,
      status: r.status, method: r.method || "Manual"
```

```
    })));
    return res.status(201).json({ success: true, data: session });
  } catch (err) {
    return res.status(500).json({ success: false, error: err.message });
  }
};
```

5.2.4 Behavior Snapshot & Summary Aggregation

Each behavior snapshot stores the most frequent behaviors as the session's primary behaviors together with per-student records. The summary endpoint uses a MongoDB aggregation pipeline that joins records to their sessions, filters by classroom, counts each behavior per student and reshapes the result into one object per student, which the Behavior Summary table displays:

```
// backend/controllers/behaviorController.js - Snapshot & Per-Student Summary
const mongoose = require("mongoose");
const BehaviorSession = require("../models/BehaviorSession");
const BehaviorRecord = require("../models/BehaviorRecord");

exports.saveSnapshot = async (req, res) => {
  const { classroomId, observations } = req.body; // [{studentId, behavior, ...}]
  const tally = {};
  observations.forEach(o => {
    tally[o.behavior] = (tally[o.behavior] || 0) + 1;
  });
  const top = Math.max(...Object.values(tally));
  const primary = Object.keys(tally).filter(k => tally[k] === top);

  const session = await BehaviorSession.create({
    classroom_id: classroomId, captured_at: new Date(),
    observed_count: observations.length, primary_behaviors: primary
  });
  await BehaviorRecord.insertMany(observations.map(o => ({
    session_id: session._id, student_id: o.studentId,
    behavior: o.behavior || "UNKNOWN", confidence: o.confidence ?? null,
    present: o.present !== false
  })));
  res.status(201).json({ success: true, data: session });
};

exports.getSummary = async (req, res) => {
  const classroomId = new mongoose.Types.ObjectId(req.params.classroomId);
  const summary = await BehaviorRecord.aggregate([
    { $lookup: { from: "behaviorsessions", localField: "session_id",
```

```
                foreignField: "_id", as: "s" } },
    { $unwind: "$s" },
    { $match: { "s.classroom_id": classroomId } },
    { $group: { _id: { student: "$student_id", behavior: "$behavior" },
                count: { $sum: 1 } } },
    { $group: { _id: "$_id.student",
                counts: { $push: { k: "$_id.behavior", v: "$count" } } } },
    { $project: { counts: { $arrayToObject: "$counts" } } },
    { $lookup: { from: "students", localField: "_id",
                 foreignField: "_id", as: "student" } },
    { $unwind: "$student" }
  ]);
  res.json({ success: true, data: summary });
};
```

5.2.5 JWT Authentication & Role Guards

All private routes are protected by two middleware functions. *requireAuth* verifies the Bearer token signed with the server secret and attaches the user to the request; *requireRole* allows the listed roles and always allows Admin:

```
// backend/middleware/auth.js - JWT Verification & Role Guard
const jwt = require("jsonwebtoken");

function requireAuth(req, res, next) {
  const header = req.headers.authorization || "";
  if (!header.startsWith("Bearer ")) {
    return res.status(401).json({ success: false, error: "Token required." });
  }
  try {
    req.user = jwt.verify(header.split(" ")[1], process.env.JWT_SECRET);
    return next();
  } catch {
    return res.status(401).json({ success: false,
      error: "Invalid or expired token." });
  }
}

function requireRole(...roles) {
  return (req, res, next) => {
    if (!req.user) return res.status(401).json({ success: false });
    if (req.user.role === "admin" || roles.includes(req.user.role)) {
      return next();
    }
    return res.status(403).json({ success: false,
      error: `Access denied for role: ${req.user.role}` });
  };
}

module.exports = { requireAuth, requireRole };
```

5.3 TESTING AND OBSERVED RESULTS

The system was tested on a laptop with a built-in webcam, using a classroom named “aiads” (section C, subject DBMS) with two enrolled students. Table 5.1

summarises the functional tests and the results observed in the portal, which are also visible in the screenshots that follow the appendix.

Table 5.1: Functional Test Cases and Observed Results

TC	Test Case	Observed Result	Status
TC-01	Create classroom with name, section and subject	“aiads” (C, dbms) listed with teacher and 2 enrolled	Pass
TC-02	Add students with roll number and classroom	Students 139 and 128 listed under aiads	Pass
TC-03	Face enrollment and live recognition	Both students shown as Face Verified / Detected in Live Camera	Pass
TC-04	Auto-fill and save attendance	Session saved: 2 present, 0 absent, 0 late, 100%	Pass
TC-05	Inspect attendance session roster	Method recorded as AI Face Recognition / Vision	Pass
TC-06	Auto-fill behavior from camera	SPEAKING (98%) and STUDYING (96%) detected	Pass
TC-07	Save and list behavior snapshots	6 sessions logged with primary behaviors	Pass
TC-08	Per-student behavior summary	Counts aggregated per behavior for each student	Pass
TC-09	Live HUD stream and sync	MJPEG stream at about 2.1 FPS; auto-sync every 10 s	Pass
TC-10	Student leaves camera frame	Logged as UNKNOWN behavior while still marked present	Pass

The tests confirm that the end-to-end flow works: students are recognised and marked present without a roll call, attendance sessions are stored with correct totals, and behavior snapshots accumulate into a meaningful per-student summary. The observed frame rate of about 2 FPS on a CPU-only laptop is sufficient for attendance and periodic behavior sampling, since classroom behavior changes over seconds rather than milliseconds; a GPU would raise the rate substantially. When no student is inside the camera frame, behavior is logged as UNKNOWN rather than guessed, which keeps the analytics honest. The main limitations observed are sensitivity to poor lighting and to students sitting far from the camera, and the small size of the test group; these are addressed in the future enhancements.

CHAPTER 6
CONCLUSION & FUTURE WORK

This chapter deals with the evaluation of Smart Classroom against its objectives, summarises the achievements of the project and outlines future enhancements.

6.1 CONCLUSION

Smart Classroom shows that a single camera and openly available deep learning models can automate classroom attendance and give teachers objective information about student engagement. By combining a React portal, an Express REST API, MongoDB and a Python vision service built on FaceNet and YOLOv8, the platform removes the need for manual roll calls, prevents proxy attendance, and records how students behave during a lecture in a form that can be reviewed later.

Key accomplishments of the project include:

- **Contactless AI Attendance:** Face-verified attendance filled from the camera in one click and saved as sessions with present, absent and late counts and attendance percentage.
- **Real-Time Behavior Monitoring:** Six behavior classes detected with confidence scores, stored as timestamped snapshots and aggregated into per-student summaries.
- **Live Vision Dashboard:** An annotated MJPEG HUD stream with live presence, behavior, FPS and faces-in-frame indicators, synchronised with the database every 10 seconds.
- **Clean Management Portal:** Classroom and student management with secure JWT authentication and role-based access.
- **Privacy-Conscious Design:** Only numeric face embeddings are stored, not face images.

6.2 FUTURE ENHANCEMENTS

1. **GPU and Edge Deployment:** Running the vision service on a GPU or an edge device such as an NVIDIA Jetson to raise the frame rate and support larger classes.

2. **Custom-Trained Behavior Model:** Collecting a labelled classroom dataset and fine-tuning YOLOv8 to recognise behaviors directly, improving accuracy over rule-based fusion.
3. **Multi-Camera Coverage:** Combining several cameras for large halls and back-row students, with tracking to avoid counting the same student twice.
4. **Alerts and Reports:** Automatic email or SMS alerts for low attendance, and weekly engagement reports for mentors and parents.
5. **University Portal Integration:** Exporting saved attendance directly to the college ERP or university attendance system through an API.

APPENDIX

SOURCE CODE:

1. Project Structure

Project folder structure of the Smart Classroom application:

```
smart-classroom/
|-- backend/                          # Node.js + Express REST API (port 5000)
|   |-- config/db.js                  # MongoDB connection (port 27017)
|   |-- models/
|   |   |-- User.js                   # Admin / teacher accounts
|   |   |-- Classroom.js              # Name, section, subject, teacher
|   |   |-- Student.js                # Roll no, name, email, classroom
|   |   |-- AttendanceSession.js      # Per-session totals and percentage
|   |   |-- AttendanceRecord.js       # Per-student status and method
|   |   |-- BehaviorSession.js        # Snapshot header and primary behaviors
|   |   `-- BehaviorRecord.js         # Per-student behavior and confidence
|   |-- controllers/                  # auth, classroom, student,
|   |                                 # attendance, behavior, vision proxy
|   |-- middleware/auth.js            # JWT verification and role guard
|   |-- routes/api.js                 # Central REST router
|   |-- server.js                     # Express bootstrap
|   `-- package.json
|-- ai_service/                       # Python computer-vision service (5001)
|   |-- app.py                        # Flask app, MJPEG HUD stream, REST
|   |-- recognition.py                # MTCNN + FaceNet enrollment/matching
|   |-- behavior.py                   # YOLOv8 pose + facial cue classifier
|   |-- sync.py                       # Pushes live status to backend (10 s)
|   `-- requirements.txt
`-- frontend/                         # React 18 + Vite SPA (port 5173)
    |-- src/
    |   |-- api/client.js             # Axios instance with JWT header
    |   |-- components/Sidebar.jsx    # Navigation menu
    |   |-- pages/
    |   |   |-- Dashboard.jsx
    |   |   |-- LiveVision.jsx        # Camera feed + live student status
```

```
    |   |   |-- Classrooms.jsx
    |   |   |-- Students.jsx
    |   |   |-- Attendance.jsx
    |   |   |-- Behavior.jsx
    |   |   `-- Login.jsx
    |   |-- App.jsx                   # Client-side routes
    |   `-- main.jsx
    `-- vite.config.js
```

2. Package Installation

Commands for installing dependencies and starting each service:

```
# Backend (Node.js + Express)
cd backend
npm install express mongoose dotenv cors helmet bcryptjs jsonwebtoken axios
npm install --save-dev nodemon
npm run dev                       # starts API on http://localhost:5000

# AI Vision Service (Python 3.10)
cd ai_service
pip install flask flask-cors opencv-python numpy torch facenet-pytorch \
            ultralytics pymongo requests
python app.py                     # starts MJPEG stream on http://localhost:5001

# Frontend (React 18 + Vite)
cd frontend
npm install react-router-dom axios
npm run dev                       # opens http://localhost:5173
```

3. Database Connection

MongoDB connection using Mongoose (backend/config/db.js):

```
// backend/config/db.js - MongoDB Connection
const mongoose = require("mongoose");

async function connectDB() {
  const uri = process.env.MONGODB_URI ||
              "mongodb://127.0.0.1:27017/smart_classroom";
  try {
    await mongoose.connect(uri, { serverSelectionTimeoutMS: 5000 });
    console.log("MongoDB connected:", mongoose.connection.host);
  } catch (err) {
    console.error("MongoDB connection failed:", err.message);
    process.exit(1);
  }
}

module.exports = connectDB;
```

4. Environment Variables

Configuration parameters stored in .env files:

```
# backend/.env  (values shown are placeholders - never commit real secrets)
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/smart_classroom
JWT_SECRET=<your-long-random-secret>
JWT_EXPIRES_IN=1d
AI_SERVICE_URL=http://localhost:5001
CORS_ORIGIN=http://localhost:5173

# ai_service/.env
CAMERA_INDEX=0
BACKEND_URL=http://localhost:5000/api
SYNC_INTERVAL_SECONDS=10
MATCH_THRESHOLD=0.70
```

5. Data Models

Mongoose schemas for students and classrooms:

```
// backend/models/Student.js - Student Schema
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema({
  roll_no:       { type: String, required: true, unique: true, trim: true },
  name:          { type: String, required: true, trim: true },
  email:         { type: String, lowercase: true, trim: true },
  classroom_id:  { type: mongoose.Schema.Types.ObjectId, ref: "Classroom" },
  face_enrolled: { type: Boolean, default: false }
}, { timestamps: { createdAt: "created_at", updatedAt: "updated_at" } });

module.exports = mongoose.model("Student", studentSchema);

// backend/models/Classroom.js - Classroom Schema
const classroomSchema = new mongoose.Schema({
  name:       { type: String, required: true, trim: true },
  section:    { type: String, trim: true },
  subject:    { type: String, trim: true },
  teacher_id: { type: mongoose.Schema.Types.ObjectId, ref: "User" }
}, { timestamps: { createdAt: "created_at" } });

module.exports = mongoose.model("Classroom", classroomSchema);
```

6. Login and JWT Generation

Authentication controller with password verification and token issuance:

```
// backend/controllers/authController.js - Login & JWT Issuance
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");

exports.login = async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ success: false,
      error: "Email and password are required." });
  }
  const user = await User.findOne({ email: email.toLowerCase() });
  if (!user || !(await bcrypt.compare(password, user.password_hash))) {
    return res.status(401).json({ success: false,
      error: "Invalid email or password." });
  }
  const payload = { id: user._id, name: user.name, role: user.role };
  const token = jwt.sign(payload, process.env.JWT_SECRET,
                         { expiresIn: process.env.JWT_EXPIRES_IN || "1d" });
  return res.json({ success: true, token, user: payload });
};
```

7. Authentication Middleware

Bearer token verification and role guards (backend/middleware/auth.js):

```
// backend/middleware/auth.js - JWT Verification & Role Guard
const jwt = require("jsonwebtoken");

function requireAuth(req, res, next) {
  const header = req.headers.authorization || "";
  if (!header.startsWith("Bearer ")) {
    return res.status(401).json({ success: false, error: "Token required." });
  }
  try {
    req.user = jwt.verify(header.split(" ")[1], process.env.JWT_SECRET);
    return next();
  } catch {
    return res.status(401).json({ success: false,
      error: "Invalid or expired token." });
  }
}

function requireRole(...roles) {
  return (req, res, next) => {
    if (!req.user) return res.status(401).json({ success: false });
    if (req.user.role === "admin" || roles.includes(req.user.role)) {
      return next();
    }
    return res.status(403).json({ success: false,
      error: `Access denied for role: ${req.user.role}` });
  };
}

module.exports = { requireAuth, requireRole };
```

8. Create, Read and Delete APIs

Classroom and student controllers:

```
// backend/controllers/classroomController.js - Create / Read / Delete
const Classroom = require("../models/Classroom");
const Student = require("../models/Student");

exports.createClassroom = async (req, res) => {
  const { name, section, subject } = req.body;
  if (!name) {
    return res.status(400).json({ success: false,
      error: "Classroom name is required." });
  }
  const room = await Classroom.create({ name, section, subject,
                                        teacher_id: req.user.id });
  res.status(201).json({ success: true, data: room });
};

exports.getClassrooms = async (req, res) => {
  const rooms = await Classroom.find().populate("teacher_id", "name").lean();
  const withCounts = await Promise.all(rooms.map(async r => ({
    ...r, enrolled: await Student.countDocuments({ classroom_id: r._id })
  })));
  res.json({ success: true, count: withCounts.length, data: withCounts });
};

exports.deleteClassroom = async (req, res) => {
  const room = await Classroom.findByIdAndDelete(req.params.id);
  if (!room) return res.status(404).json({ success: false,
                                           error: "Classroom not found." });
  await Student.updateMany({ classroom_id: room._id },
                           { $unset: { classroom_id: "" } });
  res.json({ success: true, message: "Classroom deleted." });
};

// backend/controllers/studentController.js - Add Student
exports.createStudent = async (req, res) => {
  const { roll_no, name, email, classroom_id } = req.body;
  if (!roll_no || !name) {
    return res.status(400).json({ success: false,
      error: "Roll number and full name are required." });
  }
  if (await Student.exists({ roll_no })) {
    return res.status(409).json({ success: false,
      error: "Roll number already exists." });
  }
  const student = await Student.create({ roll_no, name, email,
                                         classroom_id: classroom_id || null });
  res.status(201).json({ success: true, data: student });
};
```

9. Express Routes

Central routing table with security middleware (backend/routes/api.js):

```
// backend/routes/api.js - Central REST Router
const router = require("express").Router();
const { requireAuth, requireRole } = require("../middleware/auth");
const auth = require("../controllers/authController");
const rooms = require("../controllers/classroomController");
const students = require("../controllers/studentController");
const attendance = require("../controllers/attendanceController");
const behavior = require("../controllers/behaviorController");
const vision = require("../controllers/visionController");

router.post("/auth/login",  auth.login);

router.get("/classrooms", requireAuth, rooms.getClassrooms);
router.post("/classrooms", requireAuth, requireRole("teacher"),
            rooms.createClassroom);
router.delete("/classrooms/:id",  requireAuth,  requireRole("admin"),
              rooms.deleteClassroom);

router.get("/students", requireAuth, students.getStudents);
router.post("/students", requireAuth, requireRole("teacher"),
            students.createStudent);
router.delete("/students/:id", requireAuth, requireRole("admin"),
              students.deleteStudent);

router.post("/attendance", requireAuth, attendance.saveAttendance);
router.get("/attendance/:classroomId", requireAuth, attendance.getSessions);
router.post("/attendance/auto-fill",  requireAuth,  vision.autoFillAttendance);

router.post("/behavior", requireAuth, behavior.saveSnapshot);
router.get("/behavior/:classroomId/sessions",  requireAuth,  behavior.getSessions);
```

```
router.get("/behavior/:classroomId/summary",  requireAuth,  behavior.getSummary);

module.exports = router;
```

10. Main Server File

Express application bootstrap (backend/server.js):

```
// backend/server.js - Express Application Bootstrap
require("dotenv").config();
const express = require("express");
const cors = require("cors");
const helmet = require("helmet");
const connectDB = require("./config/db");

const app = express();
app.use(helmet());
app.use(cors({ origin: process.env.CORS_ORIGIN }));
app.use(express.json({ limit: "2mb" }));
app.use("/api", require("./routes/api"));

app.get("/api/health", (req, res) =>
  res.json({ db: "connected", port: 27017, storage: "persistent disk" }));

const PORT = process.env.PORT || 5000;
connectDB().then(() =>
  app.listen(PORT, () => console.log(`API running on port ${PORT}`)));
```

11. AI Vision Service

Flask service with background vision loop and MJPEG stream (ai_service/app.py; helper functions load_gallery and draw_hud omitted for brevity):

```
# ai_service/app.py - Flask Vision Service with MJPEG HUD Stream
import time, threading, cv2
from flask import Flask, Response, jsonify, request
from flask_cors import CORS
from recognition import embed_faces, identify, load_gallery
from behavior import analyse_frame

app = Flask(__name__)
CORS(app)
cap = cv2.VideoCapture(0)
state = {"classroom": None, "present": {}, "behaviors": {}, "fps": 0.0,
         "in_frame": 0, "frame": None, "paused": False}
lock = threading.Lock()


def vision_loop():
    tracker, t0, n = {}, time.time(), 0
    while True:
        if state["paused"] or state["classroom"] is None:
            time.sleep(0.2); continue
        ok, frame = cap.read()
        if not ok:
            continue
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        boxes, vecs = embed_faces(rgb)
        matches = identify(vecs, load_gallery(state["classroom"]))
        acts = analyse_frame(frame, tracker)
        with lock:
            for (sid, score), act in zip(matches, acts):
                if sid:
```

```
                    state["present"][sid] = score
                    state["behaviors"][sid] = act
            state["in_frame"] = len(boxes)
            n += 1
            state["fps"] = round(n / (time.time() - t0), 1)
            state["frame"] = draw_hud(frame, boxes, matches, acts, state)


@app.route("/video_feed")
def video_feed():
    def gen():
        while True:
            with lock:
                f = state["frame"]
            if f is not None:
                _, jpg = cv2.imencode(".jpg", f, [cv2.IMWRITE_JPEG_QUALITY, 85])
                yield (b"--frame\r\nContent-Type: image/jpeg\r\n\r\n"
                       + jpg.tobytes() + b"\r\n")
            time.sleep(0.05)
    return  Response(gen(),  mimetype="multipart/x-mixed-replace;  boundary=frame")


@app.route("/vision/presence")
def presence():
    state["classroom"] = request.args.get("classroom")
    with lock:
        return jsonify({"present": list(state["present"].keys()),
                        "behaviors": state["behaviors"],
                        "fps": state["fps"], "in_frame": state["in_frame"]})
```

```
if __name__== "__main__":
    threading.Thread(target=vision_loop, daemon=True).start()
    app.run(host="0.0.0.0", port=5001, threaded=True)
```

12. React Live Vision Page

Live camera page with classroom selector and 10-second auto-sync (frontend/src/pages/LiveVision.jsx):

```
// frontend/src/pages/LiveVision.jsx - Live Camera Page with Auto-Sync
import { useEffect, useState } from "react";
import api from "../api/client";

export default function LiveVision() {
  const [rooms, setRooms] = useState([]);
  const [roomId, setRoomId] = useState("");
  const [students, setStudents] = useState([]);
  const [live, setLive] = useState({ present: [], behaviors: {}, fps: 0 });

  useEffect(() => {
    api.get("/classrooms").then(r => setRooms(r.data.data));
  }, []);

  useEffect(() => {
    if (!roomId) return;
    api.get(`/students?classroom=${roomId}`).then(r => setStudents(r.data.data));
    const poll = setInterval(async () => {            // auto-sync every 10 s
      const r = await api.get(`/vision/status?classroom=${roomId}`);
      setLive(r.data);
    }, 10000);
    return () => clearInterval(poll);
  }, [roomId]);

  const presentCount = students.filter(s => live.present.includes(s._id)).length;

  return (
    <div  className="live-page">
      <header>
        <h1>Live Computer Vision Camera</h1>
```

```
        <select value={roomId} onChange={e => setRoomId(e.target.value)}>
          <option value="">-- Select Classroom --</option>
          {rooms.map(r => <option key={r._id} value={r._id}>
            {r.name} ({r.section})</option>)}
        </select>
        <span>Present:  {presentCount}/{students.length}</span>
        <span>AI Vision FPS: {live.fps}</span>
      </header>
      <img className="feed" alt="Live feed"
           src="http://localhost:5001/video_feed" />
      <ul  className="student-list">
        {students.map(s => (
          <li key={s._id}>
            <b>{s.name}</b> Roll: {s.roll_no}
            <span>{live.present.includes(s._id) ? "PRESENT" : "ABSENT"}</span>
            <span>Behavior:  {live.behaviors[s._id]?.behavior  ||  "UNKNOWN"}</span>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

13. REST API Request and Response Examples

Sample request payloads and JSON responses:

```
// A. POST /api/students  (Header: Authorization: Bearer <token>)
{ "roll_no": "139", "name": "sabari", "email": "student@example.com",
  "classroom_id": "6718f0c2a1b2c3d4e5f60789" }

// Response: 201 Created
{ "success": true,
  "data": { "_id": "671a12f0c9e4b1a2d3e4f501", "roll_no": "139",
            "name": "sabari", "face_enrolled": false } }

// B. POST /api/attendance
{ "classroomId": "6718f0c2a1b2c3d4e5f60789",
  "records": [
    { "studentId": "671a12f0c9e4b1a2d3e4f501", "status": "Present",
      "method": "AI Face Recognition" },
    { "studentId": "671a12f0c9e4b1a2d3e4f502", "status": "Present",
      "method": "AI Face Recognition" } ] }

// Response: 201 Created
{ "success": true,
  "data": { "_id": "671b0a11d2e3f4a5b6c7d801", "present": 2, "absent": 0,
            "late": 0, "percentage": 100,
            "taken_at": "2026-10-25T00:21:38.000Z" } }

// C. GET /api/behavior/6718f0c2a1b2c3d4e5f60789/summary
{ "success": true,
  "data": [
    { "student": { "roll_no": "139", "name": "sabari" },
      "counts": { "STUDYING": 5, "UNKNOWN": 1 } },
    { "student": { "roll_no": "128", "name": "prathyuksha" },
      "counts": { "STUDYING": 2, "SPEAKING": 1, "UNKNOWN": 1 } } ] }
```

SCREENSHOTS OF THE PROJECT

The following figures show the Smart Classroom & Attendance Analytics Portal running locally with the MongoDB database connected (port 27017, persistent disk) and the AI vision service active:

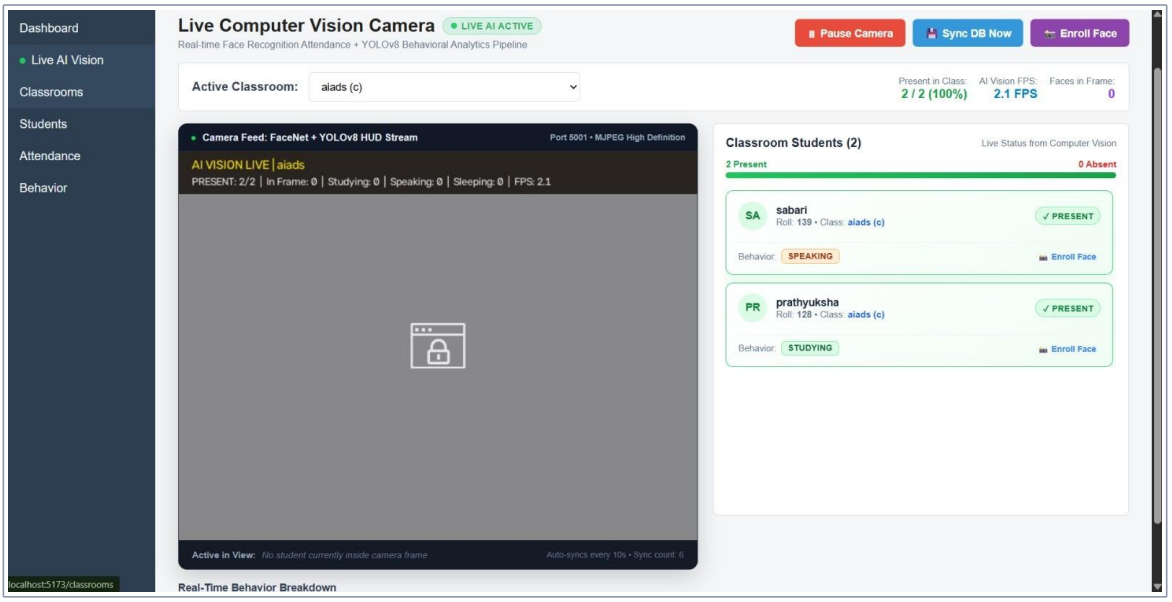


Figure 1: Live Computer Vision Camera -- FaceNet + YOLOv8 HUD Stream with Live Student Status

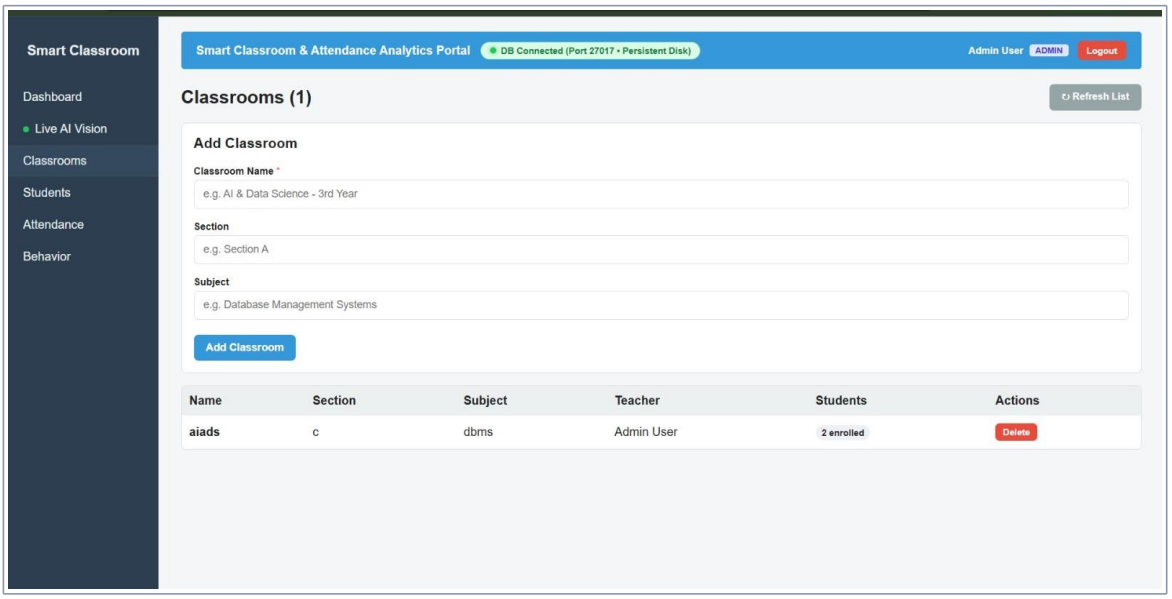


Figure 2: Classroom Management -- Add Classroom Form and Enrolled Classroom List

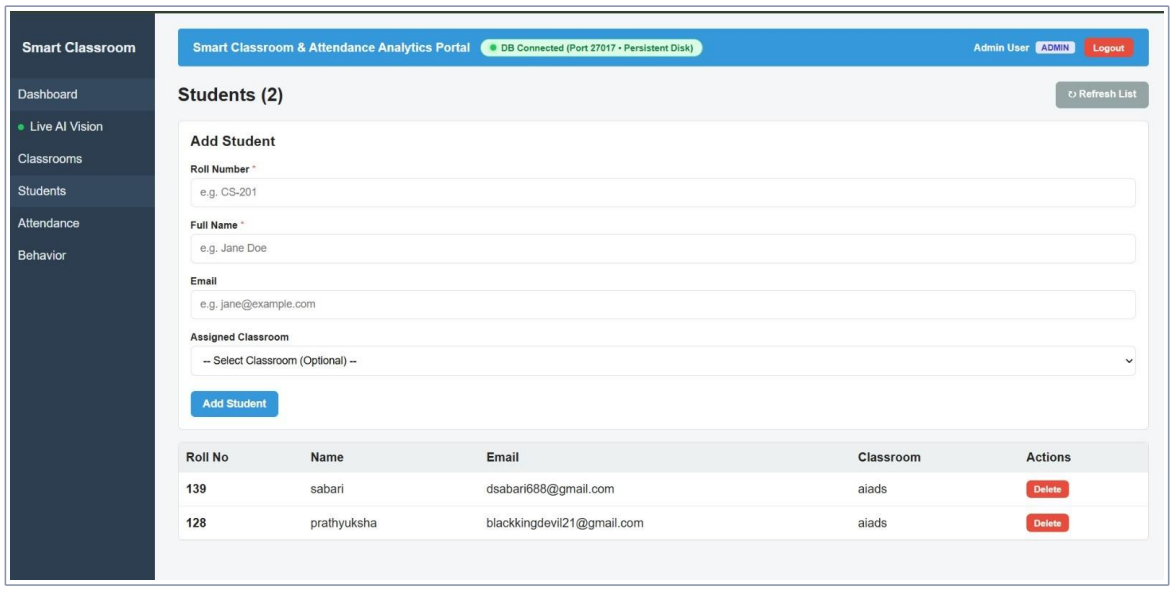


Figure 3: Student Management -- Student Registration and Classroom Assignment

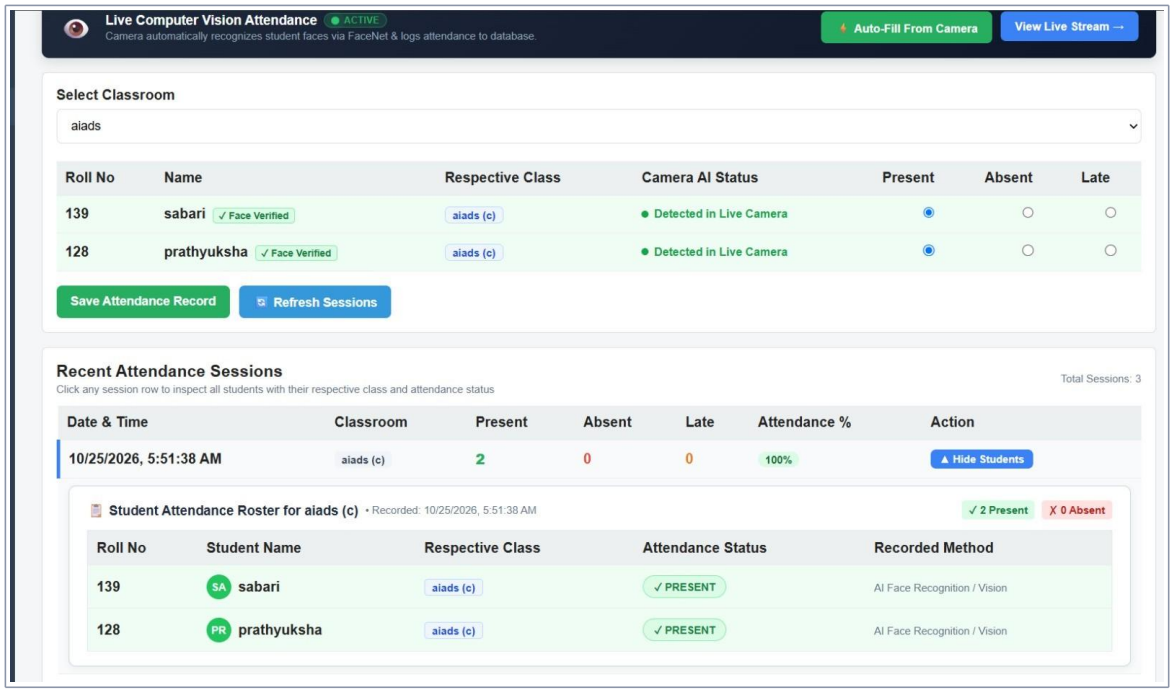


Figure 4: Live Computer Vision Attendance -- Face-Verified Roster and Recent Attendance Sessions

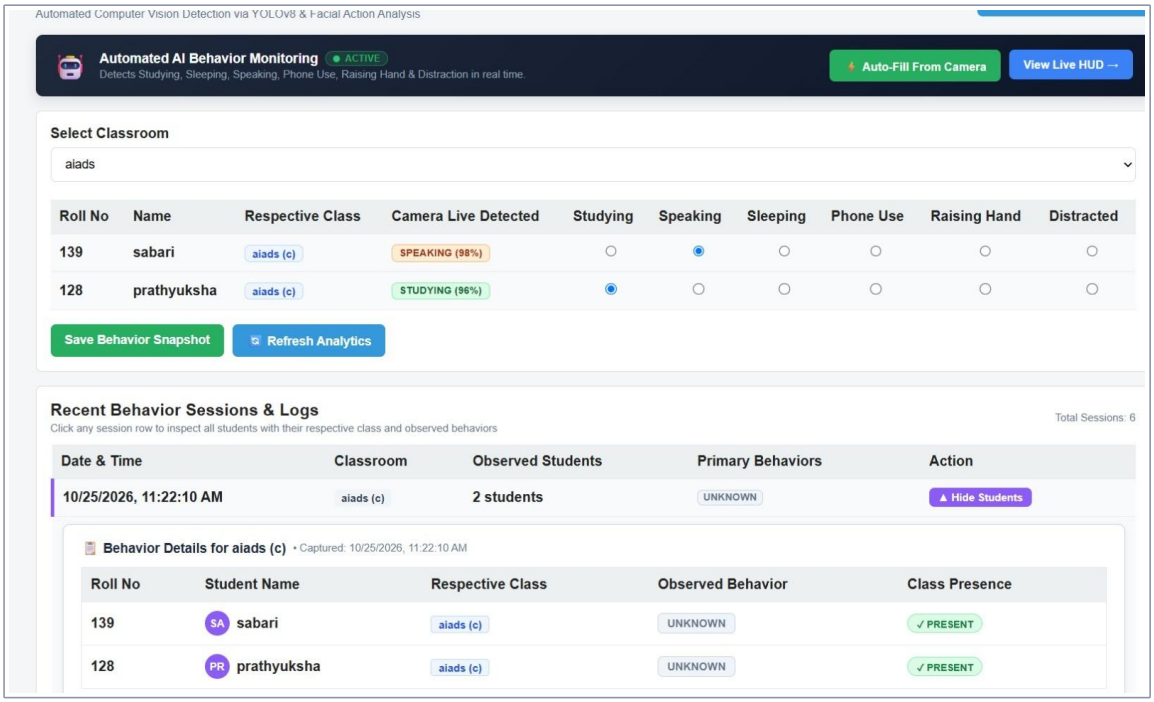


Figure 5: Automated AI Behavior Monitoring -- Camera-Detected Behaviors with Confidence Scores

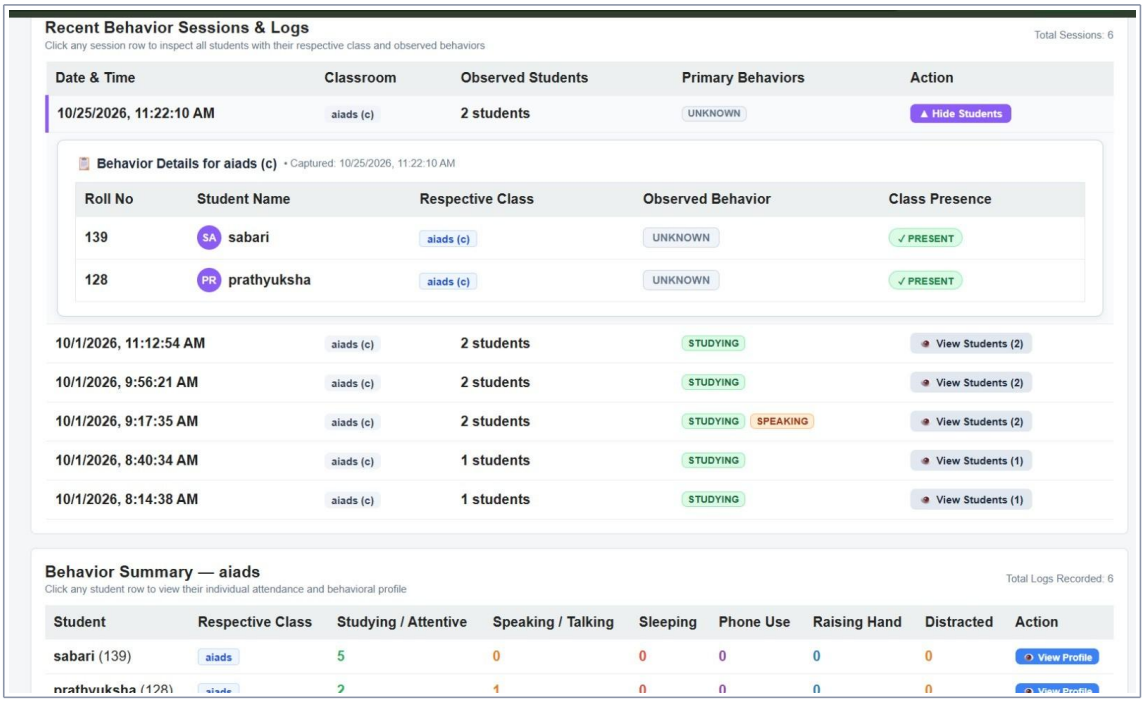


Figure 6: Recent Behavior Sessions & Logs and Per-Student Behavior Summary

REFERENCES

[1] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2001, pp. I-511–I-518.

[2] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, "Joint face detection and alignment using multitask cascaded convolutional networks," *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016.

[3] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.

[4] O. M. Parkhi, A. Vedaldi, and A. Zisserman, "Deep face recognition," in *Proc. British Machine Vision Conference (BMVC)*, 2015.

[5] Q. Cao, L. Shen, W. Xie, O. M. Parkhi, and A. Zisserman, "VGGFace2: A dataset for recognising faces across pose and age," in *Proc. 13th IEEE Int. Conf. Automatic Face & Gesture Recognition (FG)*, 2018.

[6] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, Inception-ResNet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artificial Intelligence*, 2017.

[7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[8] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[9] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 43, no. 1, pp. 172–186, 2021.

[10] T. Soukupová and J. Čech, "Real-time eye blink detection using facial landmarks," in *Proc. 21st Computer Vision Winter Workshop*, 2016.

[11] J. Whitehill, Z. Serpell, Y.-C. Lin, A. Foster, and J. R. Movellan, "The faces of engagement: Automatic recognition of student engagement from facial expressions," *IEEE Trans. Affective Computing*, vol. 5, no. 1, pp. 86–98, 2014.

[12] T. Esler, "facenet-pytorch: Face recognition using PyTorch," GitHub repository. [Online]. Available: https://github.com/timesler/facenet-pytorch

[13] OpenCV Team, "OpenCV Documentation," [Online]. Available: https://docs.opencv.org

[14] MongoDB Inc., "MongoDB Manual: Aggregation Pipelines and Data Modeling," [Online]. Available: https://www.mongodb.com/docs

[15] React Team, "React Documentation," [Online]. Available: https://react.dev

[16] OpenJS Foundation, "Express – Node.js web application framework," [Online]. Available: https://expressjs.com

[17] Pallets Projects, "Flask Documentation," [Online]. Available: https://flask.palletsprojects.com